Statistics in Health Sciences - Assignment 1

G05: Judit Félelz Guerrero (17048330), Lídia Abio Rojo (1705251), Catalina Mascaró Català (1708159), Elia Campos Villaro (1703842)

28th October 2026

1 Introduction

The aim of this assignment is to apply the framework proposed by Basagaña et al. to the data set copd in the file copd redux.RData. Data copd is a subset of 45 variables of the original data analyzed in the cited paper.

2 Analysis

2.1 Description of missingness

Provide the same missingness description than in the original analysis. Specifically, reproduce the following text that is shown in the original paper:

> "The subset used here included 342 participants and 85 variables. The range of missing data in the different variables went from 0% to 47.7%, with 68.2% of the variables having less than 5% of the data missing. Only 13.7% of the participants had complete data on all 85 variables. Overall, 5.9% of the information was missing; that is, 5.9% of the total of 29,070 cells ($85 \times 342 = 29,070$) had missing data."

Instructions:

- Note that, since you are working with a subset of the original data set, red numbers above are not expected to be the same than in your results.
- In your `Rnw` document, the text above must be able to automatically update all red numbers in the case of changing the values of `copd`. I.e., if data change, then all red numbers must also automatically update accordingly in your `pdf` when compiling your `Rnw`. As a hint, you can follow this procedure:
  1. For each red number, create an R object, which will depend on data and gives the desired quantity (i.e., if data change, the value in the object changes accordingly),
  2. To force rounding a quantity to a given decimal precision, you can use `sprintf()` (as seen in Exercise 0). Do not use `round()`,
  3. To get the thousands comma separator as in "29,070", you can use `prettyNum()`,
  4. Once you have an R object for each red number formatted as desired, use `\Sexpr{}` to insert each of them inline in LaTeX (as seen in Exercise 0).

```
> n_obs <- nrow(copd) # Number of participants
> n_vars <- ncol(copd) # Number of variables
> nans <- 100*colMeans(is.na(copd)) # Percentage of missing data by variable
> min_nan <- sprintf("%.2g", min(nans)) # MIRAR # Lower percentage
> max_nan <- sprintf("%.2g", max(nans)) # Upper percentage
>
> # Vairables with less than 5% of data missing (percentage)
> less_5 <- sprintf("%.2g", 100*length(nans[nans<5])/n_vars)
>
> # Percentage of complete cases
> comp_cases <- sprintf("%.2g", 100*mean(complete.cases(copd)))
>
> # Percentage of missing data
> miss_total <-  sprintf("%.2g", 100*mean(is.na(copd)))
>
> # Total cells observed (variables*observations)
> total_cells = prettyNum(n_obs*n_vars, big.mark = ",", decimal.mark = ".")
```

"The subset used here included 342 participants and 45 variables. The range of missing data in the different variables went from 0% to 32%, with 69% of the variables having less than 5% of the data missing. Only 33% of the participants had complete data on all 45 variables. Overall, 5.4% of the information was missing; that is, 5.4% of the total of 15,390 cells ($45 \times 342 = 15,390$) had missing data."

2.2 Multiple imputation

Apply multiple imputation to copd data using 100 imputations. Explain in a reasoned manner how you decided to set the values of the arguments `method` and `predictorMatrix` when using the `mice()` function. Show your code.

```
> # Display of default imputation methods for each variable
> ini <- mice(copd, maxit=0)
> ini

## Class: mids
## Number of multiple imputations:  5
## Imputation methods:
##                id  index_barthel2v1    quo_calcpostv1         rv_prepv1        incp_fevv1
##                ""             "pmm"                ""             "pmm"             "pmm"
##       fevp_postv1          rv_tlcv1        ctnf_s_mv1    vtdvdbid_ecov1          pp_ecov1
##                ""             "pmm"             "pmm"             "pmm"             "pmm"
##          sin11_v1           vas2_v1         ic_prepv1          iam_chv1          sin15_v1
##          "logreg"             "pmm"             "pmm"          "logreg"          "logreg"
##        cil8_s_mv1           sin7_v1           sin8_v1             ch_v1         vc_prepv1
##             "pmm"          "logreg"          "logreg"             "pmm"             "pmm"
##       dtdve_ecov1          cccs1_v0      kcals_set_v0         impactsv1          monoA_v1
##             "pmm"          "logreg"             "pmm"             "pmm"             "pmm"
##  lvmass_ase_ecov1        vtit_catv1         urg_ambv0            po2_v1           ahpf_v1
##             "pmm"          "logreg"                ""             "pmm"             "pmm"
##            pro_v1           basA_v1           fc_pmv1            tad_v1            col_v1
##             "pmm"             "pmm"             "pmm"             "pmm"             "pmm"
##           sin5_v1             bc_v1       fvcp_postv1           vas1_v1           sin6_v1
##          "logreg"             "pmm"                ""             "pmm"          "logreg"
##       graudisn_v1 cpcr_mg_l_mediav1          neutA_v1           ahpl_v1        activityv1
##         "polyreg"             "pmm"             "pmm"             "pmm"             "pmm"
## PredictorMatrix:
##                  id index_barthel2v1 quo_calcpostv1 rv_prepv1 incp_fevv1 fevp_postv1
## id                0                1              1         1          1           1
## index_barthel2v1  1                0              1         1          1           1
## quo_calcpostv1    1                1              0         1          1           1
## rv_prepv1         1                1              1         0          1           1
## incp_fevv1        1                1              1         1          0           1
## fevp_postv1       1                1              1         1          1           0
##                  rv_tlcv1 ctnf_s_mv1 vtdvdbid_ecov1 pp_ecov1 sin11_v1 vas2_v1 ic_prepv1
## id                      1          1              1        1        1       1         1
## index_barthel2v1        1          1              1        1        1       1         1
## quo_calcpostv1          1          1              1        1        1       1         1
## rv_prepv1               1          1              1        1        1       1         1
## incp_fevv1              1          1              1        1        1       1         1
## fevp_postv1             1          1              1        1        1       1         1
##                  iam_chv1 sin15_v1 cil8_s_mv1 sin7_v1 sin8_v1 ch_v1 vc_prepv1 dtdve_ecov1
## id                      1        1          1       1       1     1         1           1
## index_barthel2v1        1        1          1       1       1     1         1           1
## quo_calcpostv1          1        1          1       1       1     1         1           1
## rv_prepv1               1        1          1       1       1     1         1           1
## incp_fevv1              1        1          1       1       1     1         1           1
## fevp_postv1             1        1          1       1       1     1         1           1
##                  cccs1_v0 kcals_set_v0 impactsv1 monoA_v1 lvmass_ase_ecov1 vtit_catv1
## id                      1            1         1        1                1          1
## index_barthel2v1        1            1         1        1                1          1
## quo_calcpostv1          1            1         1        1                1          1
## rv_prepv1               1            1         1        1                1          1
## incp_fevv1              1            1         1        1                1          1
## fevp_postv1             1            1         1        1                1          1
##                  urg_ambv0 po2_v1 ahpf_v1 pro_v1 basA_v1 fc_pmv1 tad_v1 col_v1 sin5_v1
## id                       1      1       1      1       1       1      1      1       1
## index_barthel2v1         1      1       1      1       1       1      1      1       1
## quo_calcpostv1           1      1       1      1       1       1      1      1       1
## rv_prepv1                1      1       1      1       1       1      1      1       1
## incp_fevv1               1      1       1      1       1       1      1      1       1
## fevp_postv1              1      1       1      1       1       1      1      1       1
##                  bc_v1 fvcp_postv1 vas1_v1 sin6_v1 graudisn_v1 cpcr_mg_l_mediav1 neutA_v1
## id                   1           1       1       1           1                1        1
## index_barthel2v1     1           1       1       1           1                1        1
```

```
## quo_calcpostv1       1           1       1       1           1                 1        1
## rv_prepv1            1           1       1       1           1                 1        1
## incp_fevv1           1           1       1       1           1                 1        1
## fevp_postv1          1           1       1       1           1                 1        1
##                  ahpl_v1 activityv1
## id                     1          1
## index_barthel2v1       1          1
## quo_calcpostv1         1          1
## rv_prepv1              1          1
## incp_fevv1             1          1
## fevp_postv1            1          1
```

We can see that the mice package suggest imputing initialy with de mean for continuous variables and with a logistic regression for binary ones. Further research on the dataset has shown that the variables have no deterministic relation. So we do not change the imputation methods. We have a variable (`id`) that don't have any role in the imputation process, so we change `predictorMatrix` and `methods` acording to that.

```
> # Change in parameters
> meth <- ini$method
> pred <- ini$predictorMatrix
>
> meth["id"] <- "" # do not impute "id"
> pred["id", ] <- 0 # we don't want "id" to be imputed
> pred[, "id"] <- 0 # we don't want "id" to be predictor
```

```
> # Imputation of 100 datasets
> M <- 100 # Number of imputations for each missing value
> imp <- mice(data = copd,
+             m = M,
+             method = meth,
+             predictorMatrix = pred,
+             maxit = 20,
+             printFlag = FALSE,
+             seed = 666)
> # imp£imp (output llarg)
> #summary(imp)
```

2.3 Integration of multiple imputation in cluster analysis

Reproduce the original analysis in the paper using the miclust package to integrate multiple imputation in the cluster analysis. Apply the proposed framework using the same setting than in the original analysis. To do that, follow these steps:

1. Read the help of the `getdata()` function in the `miclust` package. Then, accordingly, create a *list* of *data.frames* and pass it to the `getdata()` function. Give a detailed explanation of the data processing you did to get that *list*. Show your code.

```
> # ?miclust::getdata
> # getdata() requires the data to be a list or data.frame where the first element is the raw data:
>
>   #1- prepare raw data (all numeric and without id)
> copd_raw <- copd
> copd_raw$id <- NULL
> num_cols <- sapply(copd_raw, function(x) is.numeric(x) || is.integer(x) || is.logical(x))
> copd_raw <- copd_raw[, num_cols]
>
>   #2- prepare imputated data
> imp_list <- complete(imp, action = "all") # extracts the completed data from a mids object
> imp_list_clean <- lapply(imp_list, function(df) {
+   df$id <- NULL
+   df <- df[, num_cols]
+   return(df)
+ })
```

```
>
>   #3- create list where the first element is the raw data
> my_cluster_list <- c(list(copd_raw), imp_list_clean)
>
> # prepare the data for miclust
> imp_midata <- miclust::getdata(data = my_cluster_list)
>
> # now we check
> class(imp_midata)

## [1] "list"   "midata"

> length(imp_midata$impdata)

## [1] 100
```

The `getdata()` function needs a `data.frames` list where the first element contains the original data (with missing values) and the remaining ones correspond to multiple imputed datasets. That is why we created `my_cluster_list`, the list of data frames that we pass to `getdata()`. The imputed data sets were extracted from the `mids` object, and the same processing was applied to the original data and to each imputed data set, so that all of them have the same variables in the same order. Since k-means groups the subjects using Euclidean distances, we removed the variables that are neither numeric nor logical. We also removed the `id` variable, because it is just an identifier and does not contain any valuable clinical information. For the same reason, the `id` was not included as a predictor in the imputation.

2. Use the `miclust()` function to perform the integration of missing data in cluster analysis using the same settings as in the original analysis. Show your code and explain how and why you set all arguments in the `miclust()` function.

```
> miclust_1 <- miclust(data = imp_midata,
+                      method = "kmeans",
+                      search = "backward",
+                      ks = 2:5,
+                      usedimp = 1:100,
+                      seed = 666)

## . . . . imp   5 . . . . imp  10 . . . . imp  15 . . . . imp  20
## . . . . imp  25 . . . . imp  30 . . . . imp  35 . . . . imp  40
## . . . . imp  45 . . . . imp  50 . . . . imp  55 . . . . imp  60
## . . . . imp  65 . . . . imp  70 . . . . imp  75 . . . . imp  80
## . . . . imp  85 . . . . imp  90 . . . . imp  95 . . . . imp 100
##
##  Analysis done.

> #summary(miclust_1)
> miclust_1$kfin # optimal k (number of clusters)

## [1] 2
```

We need to perform a variable selection because adding variables can degrade the classification when the number of subjects is small compared to the numer of variables. As mentioned in the article, it is good practice that the number of subjects are at least 10 times the number of variables. In our case, the ratio is $342/44 < 10$, so a reduction of the number of variables is needed.

As in the original analysis, we use the backward search with CritCF criterion. It starts with all the variables and, at each step, removes the variable that when excluded gives the highest CritCF. This criterion allows to rank partitions with different numbers of clusters and different numbers of variables. Finally, we define `k = 2:5` where the maximum value was fixed a priori, the same way as the article. Since the backward seach is repeated for each k in each imputed data set, limiting the number of clusters reduces the computational cost and also keeps the resulting groups interpretable.

3 Results

Using `plot()` and `summary()` methods in the `miclust` package, provide plots and tables for the results similar than those shown in the original paper. Also, show a barplot for the percentage of times each variable is selected among the imputed data sets, in decreasing order. Explain how you set all arguments in all functions and methods you use. Show your code.

```
> # Final number of clusters selected by miclust
> kfin <- miclust_1$kfin
>
> # Number of imputations
> n_imp <- length(miclust_1$usedimp)
```

In this section, we reproduce the figures and tables of Basagaña et al. using the `plot()` and `summary()` methods of the `miclust` object, named `miclust_1`. Following the instructions, we use one colour for all plots related to the number of clusters, dark magenta, and another for all plots related to variable selection, sky blue.

```
> # Distribution of the CritCF criterion across the imputed data sets, one box per
> # explored number of clusters.
> plot(miclust_1, metric = "critcf", col.critcf = "darkmagenta")
```

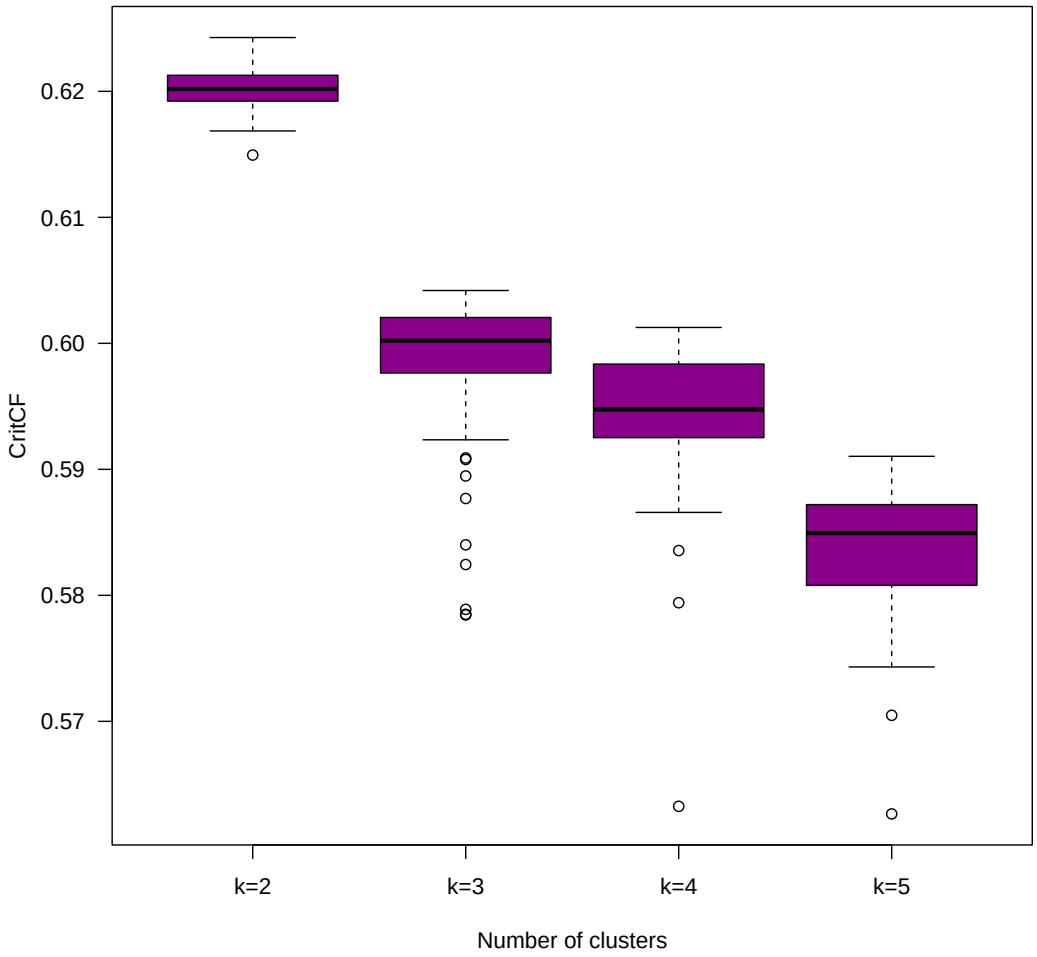


Figure 1: Distribution of the CritCF criterion (higher values are preferred) by number of clusters (k), across the 100 imputed data sets. Each box summarises 100 values obtained with k-means and backward variable selection for $k = 2, ..., 5$. The final number of clusters, $k = 2$, is the one selected in most imputed data sets.

Figure 1 corresponds to Figure 1 of the original paper. It is obtained with `plot(miclust_1, metric = "critcf")`, the argument `metric = "critcf"` selects the boxplots of the CritCF criterion by numbers of clusters, with one value per imputed data set and per k. The argument `col.critcf = "darkmagenta"` makes de boxes in color.

```
> # Percentage of appearance of each variable (metric = "varsel") for k = kfin.
> plot(miclust_1, k = kfin, metric = "varsel", col.varsel = "skyblue")
```

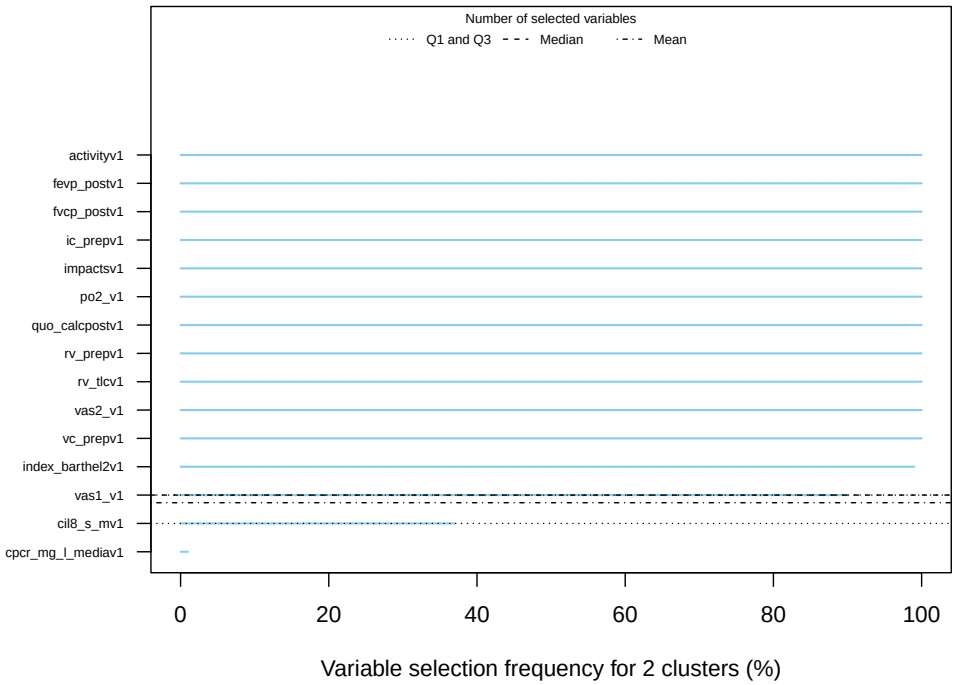


Figure 2: Variable selection frequency for $k = 2$ clusters. Each horizontal line shows the percentage of the 100 imputed data sets in which a variable remained in the final set after the backward selection algorithm. Only variables selected in at least one data set are shown. The black horizontal lines summarise the number of variables selected per imputed data set: first and third quartiles (dotted), median (dashed) and mean (dash-dotted), positioned according to the ranked list of variables. Variable names are those of the `copd` data set.

Figure 2 corresponds to Figure 2 of the original paper. We use `metric = "varsel"` to show only, for each variable, the percentages of imputed data sets in which it remained in the final set of variables. With `k = kfin` we show the results for the optimal number of clusters, as in the paper, where only the data sets with the final k are used for variable selection. The argument `col.varsel = "skyblue"` sets the color of the segments.

```
> # Frequency of selection of each variable across the imputed data sets.
> y   <- getvariablesfrequency(miclust_1)
>
> # Sort the variables by decreasing percentage of selection.
> y_ord <- order(y$percfreq, decreasing = TRUE)
> perc <- y$percfreq[y_ord]
> name <- y$varnames[y_ord]
>
> # Bars are drawn as thick vertical lines (type = "h", lwd = 20).
> plot(perc, type = "h", main = "", xlab = "", ylab = "Percentage of times selected",
+      xlim = 0.5 + c(0, length(perc)), ylim = c(0, 100),
+      lwd = 20, col = "skyblue", xaxt = "n")
>
> # Variable names below each bar: las = 2 draws them vertically, and
> # cex.axis = 0.45 reduces their size so that the long names fit.
> axis(1, at = 1:length(name), labels = name, las = 2, cex.axis = 0.45)
```

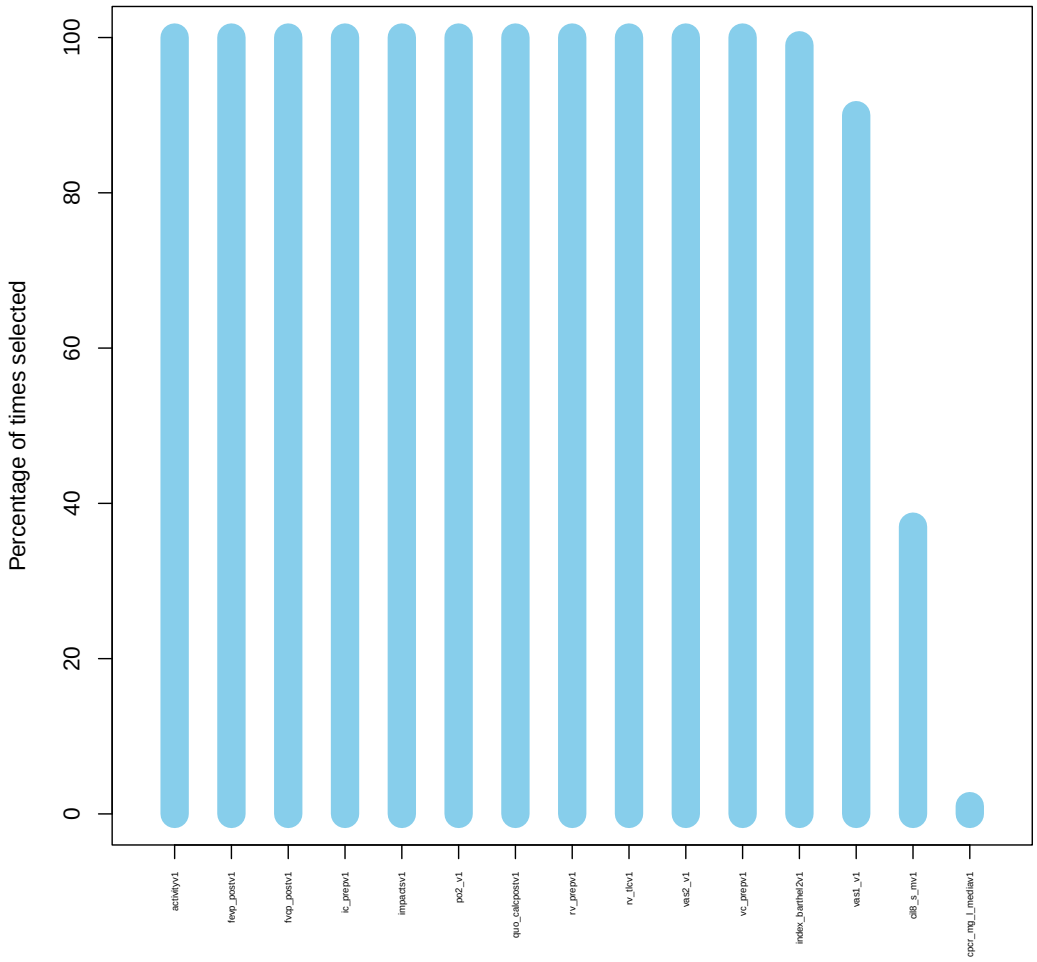


Figure 3: Percentage of the 100 imputed data sets in which each variable was selected by the backward algorithm for $k = 2$, in decreasing order. Only variables selected in at least one imputed data set are shown. Bar heights represent the percentage of selection.

```
> # This code is adapted from the miclust README:
> # https://cran.r-project.org/web/packages/miclust/readme/README.html
```

Figure 3 shows the same selection percentages as Figure 2, as a barplot in decreasing order. The percentages are obtained with `getvariablesfrequency(miclust_1)`, which returns the variable names (`varnames`) and the percentage of times each one was selected (`percfreq`). They are sorted with `order(..., decreasing = TRUE)`, applying the same ordering to the percentages and to the names so that each bar keeps its label. The bars are drawn with `plot(type = "h")` and a thick line (`lwd = 20`). The default x axis is removed (`xaxt = "n"`) and replaced by `axis()`, with vertical labels (`las = 2`) and reduced size (`cex.axis`) so that long names fit. The y axis is fixed to `ylim = c(0, 100)` because the values are percentages, and `col = "skyblue"` keeps the color used for variable selection. The code is adapted from the `miclust` README.

```
> # Number of clusters used for comparison.
> #   kfin + 1, if kfin < max(miclust_1£ks)
> #   kfin - 1 else.
> if (kfin < max(miclust_1$ks)) {
+   k_comp <- kfin + 1
+ } else {
```

```
+   k_comp <- kfin - 1
+ }
>
> # Set quantilevars = 0.5 so the numbers of final variables equals the median
> # number of variables selected across the imputed data sets.
> summary_k <- summary(miclust_1, quantilevars = 0.5)
> summary_kp1 <- summary(miclust_1, k =  k_comp, quantilevars = 0.5)
```

The tables are built from the `summary()` method. We call `summary(miclust_1, quantilevars = 0.5)`, which uses the optimal number of clusters ($k = 2$). The argument `quantilevars = 0.5` makes the number of final variables equal to the median number of variables selected across the imputed data sets, and the variables retained are the most frequently selected ones, which is the criterion proposed in step 3a of Table 1 of Basagaña et al. We also call `summary()` with `k = k_comp` ($k = 3$) only for comparison. The value `k_comp` is $k_{\text{fin}} + 1$, unless k_{fin} is the largest explored number of clusters (5), in which case we use $k_{\text{fin}} - 1$.

```
> # Distribution of the assignment probabilities for each cluster,
> # for k = kfin.
> a_fin <- as.matrix(summary_k$allocationprobabilities)
>
> # Same distribution, for k = k_comp.
> a_comp <- as.matrix(summary_kp1$allocationprobabilities)
>
> # Order the blocks by increasing number of clusters.
> if (k_comp > kfin) {
+   k1 <- kfin
+   a1 <- a_fin
+   k2 <- k_comp
+   a2 <- a_comp
+ } else {
+   k1 <- k_comp
+   a1 <- a_comp
+   k2 <- kfin
+   a2 <- a_fin
+ }
>
> # Numeric part: an empty row before each k block, followed by the rows
> # of the corresponding summary.
> vals <- rbind(rep(NA, 5), a1, rep(NA, 5), a2)
>
> # Text part: one label per row.
> labels <- c(paste0("$k=", k1, "$"),
+             paste0("Cluster ", seq_len(nrow(a1))),
+             paste0("$k=", k2, "$"),
+             paste0("Cluster ", seq_len(nrow(a2))))
>
> # Combine the text labels in the first column with the numeric values.
> tab_alloc <- data.frame(labels, vals)
>
>
> # Create the xtable object. 'digits' has seven elements: the first
> # specifies the automatic row names (not printed), the second specifies
> # the text labels, and the remaining five specify the numeric columns.
> # Define the caption and label.
> alloc <- xtable(tab_alloc,
+                 digits = c(0, 0, 2, 2, 2, 2, 2),
+                 align = c("l", "l", "c", "c", "c", "c", "c"),
```

```
+                 caption = paste0(
+                   "Distribution of the assignment frequencies of participants ",
+                   "to each cluster across the ", length(miclust_1$usedimp),
+                   " imputed data sets, for $k=", k1, "$ and $k=", k2, "$ ",
+                   "(the final number of clusters is $k=", kfin, "$; ",
+                   "the other value is shown for comparison only). ",
+                   "For each participant, the assignment frequency for a cluster ",
+                   "is the proportion of imputed data sets in which the participant ",
+                   "was assigned to that cluster. ",
+                   "Each row gives the minimum, first quartile, median, third ",
+                   "quartile, and maximum of these frequencies among participants ",
+                   "finally assigned to that cluster by majority vote."),
+                 label = "tab:alloc")
>
> # Column names, centred and in bold; the first column is left blank.
> colnames(alloc) <- c("","\\multicolumn{1}{c}{\\bf Minimum}",
+                          "\\multicolumn{1}{c}{\\bf First quartile}",
+                          "\\multicolumn{1}{c}{\\bf Median}",
+                          "\\multicolumn{1}{c}{\\bf Third quartile}",
+                          "\\multicolumn{1}{c}{\\bf Maximum}")
>
> # Print the table as LaTeX code.
> print(alloc,
+       table.placement = "ht",
+       size = "footnotesize",
+       include.rownames = FALSE, # Row names are not printed, labels are in columne 1
+       include.colnames = TRUE,
+       floating = TRUE,
+       sanitize.text.function = force,
+       booktabs = TRUE)
```

	Minimum	First quartile	Median	Third quartile	Maximum
$k = 2$					
Cluster 1	0.50	1.00	1.00	1.00	1.00
Cluster 2	0.57	1.00	1.00	1.00	1.00
$k = 3$					
Cluster 1	0.64	1.00	1.00	1.00	1.00
Cluster 2	0.56	1.00	1.00	1.00	1.00
Cluster 3	0.78	1.00	1.00	1.00	1.00

Table 1: Distribution of the assignment frequencies of participants to each cluster across the 100 imputed data sets, for $k = 2$ and $k = 3$ (the final number of clusters is $k = 2$; the other value is shown for comparison only). For each participant, the assignment frequency for a cluster is the proportion of imputed data sets in which the participant was assigned to that cluster. Each row gives the minimum, first quartile, median, third quartile, and maximum of these frequencies among participants finally assigned to that cluster by majority vote.

Table 1 corresponds to Table 2 of the original paper. It uses the field `allocationprobabilities` of the summaries for $k = 2$ and $k = 3$, which gives the minimum, quartiles and maximum of the frequency of assignment of the participants to their final cluster over the 100 imputed data sets. A block of rows is built for each k, always in increasing order of k, so the table adapts if the number of clusters changes.

```
> # Final selected variables and the cluster summary.
> vars <- summary_k$selectedvariables
> summary_cluster <- summary_k$summarybycluster
>
```

```
> # Percentage of missing values for each variable.
> miss <- summary_cluster$`%miss.`
>
> # Imputed data means for each variable by cluster.
> mean_imputed <- as.matrix(
+   summary_cluster[, grep("^mean", names(summary_cluster))])
>
> # Majority-vote cluster assignment for each participant.
> cl <- summary_k$cluster
>
> # Empty matrix (variables x clusters) for the raw-data means, and empty vector
> # for the raw-data F values.
> raw_means <- matrix(NA, nrow = length(vars), ncol = kfin)
> F_raw <- vector(length = length(vars))
>
> # Raw data: compute the F-statistic and mean for each selected variable
> # in each cluster.
> for (i in 1:length(vars)) {
+       # Values of the selected variable for all participants.
+       x <- copd_raw[[vars[i]]]
+       # Fit a linear model with cluster as the explanatory factor.
+       fit <- lm(x ~ factor(cl))
+       # Extract the F-statistic for the cluster effect from the first row of
+       # the ANOVA table.
+       F_raw[i] <- anova(fit)$`F value`[1]
+   # Mean per cluster, excluding NAs to avoid getting NA values.
+   for (j in 1:kfin){
+     raw_means[i,j] <- mean(x[cl == j], na.rm = TRUE)
+   }
+ }
>
> # Cluster assignments for each participant in each imputed data set
> # (participants x imputed data sets), and the number of imputed data sets.
> vector_clusters <- summary_k$clustervectors
> n_imp <- ncol(vector_clusters)
>
> # Empty vector for the F values of the imputed data.
> F_imputed <- vector(length = length(vars))
>
> # Imputed data: same as for the raw data, but repeated in each imputed data set
> # using its own cluster assignment, the F values are then averaged.
> for (i in 1:length(vars)){
+   f_n_imp <- vector(length = n_imp)
+   for (m in 1:n_imp){
+     y <- imp_midata$impdata[[m]][[vars[i]]]
+     fit <- lm(y ~ factor(vector_clusters[, m]))
+     f_n_imp[m] <- anova(fit)$`F value`[1]
+   }
+   F_imputed[i] <- mean(f_n_imp)
+ }
>
> # Cluster sizes in the raw data and median cluster sizes across imputed data sets.
> n_raw     <- as.vector(table(factor(cl, levels = seq_len(kfin))))
> n_imp_med <- summary_k$clusterssize[, "size"]
>
```

```
> # Minimum and maximum cluster size overs the imputed data sets, for the caption.
> range_txt <- paste0("cluster ", seq_len(kfin), ": ",
+                     summary_k$clusterssize[, "min."], "--",
+                     summary_k$clusterssize[, "max."], collapse = "; ")
>
> # Escape underscores in variable names for LaTeX.
> labels <- gsub("_", "\\_", vars)
>
> # Create the final data frame, with variable labels followed by the numerical
> # results.
> tab_desc <- data.frame(labels, miss, raw_means, F_raw, mean_imputed, F_imputed)
>
> # Create the xtable object. The caption and the label are defined here.
> desc <- xtable(tab_desc,
+                digits = c(0, 0, 1, rep(2, kfin), 2, rep(2, kfin), 2),
+                align  = c("l", "l", rep("c", 2 * kfin + 3)),
+                caption = paste0(
+                  "Mean values of the ", length(vars),
+                  " selected variables by cluster ($k=", kfin, "$). ",
+                  "Raw data: majority-vote cluster assignment, ",
+                  "missing values excluded. ",
+                  "Imputed data: means over the ", n_imp,
+                  " imputed data sets. ",
+                  "$n$: number of participants per cluster (raw data: ",
+                  "majority-vote assignment, imputed data: means over ",
+                  "the imputed data sets, with minimum--maximum ",
+                  range_txt, "). ",
+                  "$F$: ratio of between cluster variance ",
+                  "from a linear regression of each variable on cluster ",
+                  "(imputed data: mean over the imputed data sets). ",
+                  "\\% missing: percentage of participants with missing values."),
+                label = "tab:desc")
>
> # Two-level header, first row have Raw data and Imputed data, the second row have
> # variable, % missing, clusters with their sizes, and F values.
> head_top <- paste0(
+             "& & \\multicolumn{", kfin + 1, "}{c}{\\bf Raw data} & ",
+             "\\multicolumn{", kfin + 1, "}{c}{\\bf Imputed data} \\\\\n")
> head_sub <- paste0(
+             "\\bf Variable & \\bf \\% missing & ",
+             paste0("\\bf Cl.~", 1:kfin, " ($n=", n_raw, "$)", collapse = " & "),
+             " & \\bf $F$ & ",
+             paste0("\\bf Cl.~", 1:kfin, " ($n=", n_imp_med, "$)", collapse = " & "),
+             " & \\bf $F$ \\\\\n")
>
> # Create the cmidrule lines.
> cmidrules <- paste0("\\cmidrule(lr){3-", kfin + 3, "} ",
+                     "\\cmidrule(lr){", kfin + 4, "-", 2 * kfin + 4, "}\n")
>
> # Store the complete add.to.row structure in a variable.
> head_structure <- list(pos=list(0),
+                        command = paste0(head_top, cmidrules, head_sub))
>
> # Print the table as LaTeX code.
> print(desc,
```

```
+       table.placement = "H",
+       size = "footnotesize",
+       include.rownames = FALSE,
+       include.colnames = FALSE,          # header is written by add.to.row
+       add.to.row = head_structure,
+       floating = TRUE,
+       sanitize.text.function = force,
+       booktabs = TRUE)
```

		Raw data			Imputed data		
Variable	**% missing**	**Cl. 1** ($n = 184$)	**Cl. 2** ($n = 158$)	F	**Cl. 1** ($n = 184$)	**Cl. 2** ($n = 158$)	F
activityv1	1.2	37.21	58.86	85.26	37.01	59.02	88.56
fevp_postv1	0.0	62.90	39.09	359.64	62.87	39.08	353.71
fvcp_postv1	0.0	79.74	63.48	112.55	80.08	63.66	115.07
ic_prepv1	5.6	73.50	52.05	187.30	73.31	51.93	205.82
impactsv1	1.2	18.54	35.16	80.23	18.75	35.61	79.24
po2_v1	3.2	77.66	69.70	52.98	77.21	69.93	55.33
quo_calcpostv1	0.0	58.96	46.12	140.01	58.80	46.03	134.83
rv_prepv1	7.9	136.19	187.69	122.19	136.25	186.76	124.53
rv_tlcv1	7.9	50.26	63.59	266.57	50.43	63.59	280.59
vas2_v1	1.2	4.23	6.34	56.62	4.24	6.43	58.50
vc_prepv1	5.6	78.00	60.59	160.56	77.87	60.11	180.36
index_barthel2v1	2.6	99.89	98.32	14.83	99.93	98.31	15.99
vas1_v1	1.2	1.99	3.99	58.60	1.96	4.05	59.22

Table 2: Mean values of the 13 selected variables by cluster ($k = 2$). Raw data: majority-vote cluster assignment, missing values excluded. Imputed data: means over the 100 imputed data sets. n: number of participants per cluster (raw data: majority-vote assignment, imputed data: means over the imputed data sets, with minimum–maximum cluster 1: 179–189; cluster 2: 153–163). F: ratio of between cluster variance from a linear regression of each variable on cluster (imputed data: mean over the imputed data sets). % missing: percentage of participants with missing values.

Table 2 corresponds to Table 3 of the original paper and describes the 13 final variables by cluster. The percentage of missing values and the means for the imputed data come from the field `summarybycluster`, and the cluster sizes (median, minimum and maximum over the imputed data sets) from `clusterssize`. For the raw data, each participant is assigned to the cluster given by majority vote (`summary_k$cluster`) and the mean of each variable by cluster is computed with `mean(..., na.rm = TRUE)`, so that missing values are excluded. The F values are obtained with `lm()` of each variable on the cluster, as a `factor`, so that the cluster numbers are treated as labels, and `anova()`, taking the F of the first row. For the imputed data, this is repeated in each of the 100 imputed data sets with its own cluster assignment (`clustervectors`), and the F values are averaged. The article does not specify how to combine them, so the mean is our choice.

4 Report

Write a brief report (400 words maximum) describing the following issues, and citing tables and figures:

- The final number of clusters and their sizes and uncertainty (cite your figure(s) and table(s)).
- The agreement when relabeling the clusters.
- The final subset of clustering variables and uncertainty.
- The probability of assignment to each cluster and uncertainty.

To find the number of clusters, we applied the backward search with the CritCF criterion to each of the 100 imputed data sets. Figure 1 shows that even the worst value for $k = 2$ is above the best value for $k = 3$, so $k = 2$ was the optimal number of clusters in all the imputed data sets. Therefore, there

is practically no uncertainty in the choice of k. After the assignation, the clusters contain 184 and 158 subjects.

To prevent a potential problem caused by the automatic labeling of the clusters, the clusters were relabeled taking the first data set as a reference and evaluating them with Cohen's kappa, same as the article. The agreement after relabeling is good, with a median Cohen's kappa of 0.97.

For $k = 2$, 15 of 33 variables candidates were selected in at least one data set(Figure 2 and 3). The final number of selected variables is 13, with 11 of those that are in all the imputed data sets. There is a clear drop in frequency right after the last selected variable (from 90% to 37%). Also, we see that the selected variables would be the same if we kept those appearing in more than 50% of the data sets. Therefore, the variable selection is very robust to the missing data.

Each subject was assigned to the cluster to which they were allocated most often across the imputed data sets. In table 1 we can see how this frequency is distributed in $k = 2$ and $k = 3$. For $k = 2$, the median probability is 1.00 and 1.00 in both clusters, so most subjects are always assigned to the same cluster. For $k = 3$, the assignment is also very stable, and this is probably because the selected variables have few missing data, so the imputed data sets are very similar. Given this, we see that a stable assignment does not necessarily mean that the clusters are adequate, we have already seen that the choice of $k = 2$ is supported by the CritCF criterion. Table 2 shows that cluster 2 groups the subjects with more severe disease with worse lung function, more air trapping, lower arterial oxygen and worse scores in the questionnaires. We also see that results with raw and imputed data are very similar.